Low-Rank Approximation and Image Compression

MA26M009 (Team Leader) MA26M010 MA26M011 MA26M012

Theory

An image is a matrix

A grayscale image is just a table of numbers. Each number is the brightness of one pixel, from 0 (black) to 255 (white). An image with m rows and n columns is therefore an $m \times n$ matrix A. A colour image is first turned into gray with gray $= 0.299R + 0.587G + 0.114B$, which roughly matches how bright the colours look to the eye. This way we only have one matrix per image.

The SVD

The singular value decomposition writes any matrix as a product $A = U\Sigma V^\top$. Here Σ is a diagonal matrix holding the singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \geq 0$, where $k = \min(m, n)$, and the columns u_i of U and v_i of V are orthonormal. Written as a sum this is

$$A = \sum_{i=1}^{k} \sigma_i \, u_i v_i^\top .$$

Each term is a column u_i times a row $v_i^\top$, scaled by σ_i. We can think of the terms as layers of the image. The first layers hold the big shapes and the overall brightness, and the later ones add smaller and smaller details. The singular value σ_i tells us how important layer i is.

Low-rank approximation

If we keep only the first r layers, we get the rank-r approximation

$$A_r = \sum_{i=1}^{r} \sigma_i \, u_i v_i^\top = U_r \Sigma_r V_r^\top .$$

One layer $u_i v_i^\top$ has a very simple look: every row is a scaled copy of the same row $v_i^\top$. So images built from only a few layers show horizontal and vertical streaks. This is what we see for small r in the results.

The Eckart–Young theorem says that no other matrix of rank r is closer to A than A_r (in the Frobenius norm).

$$\|A - A_r\|_{\mathrm{F}} = \Big(\sum_{i>r} \sigma_i^2 \Big)^{1/2} .$$

Relative error and energy

Since $\|A\|_{\mathrm{F}}^2 = \sum_i \sigma_i^2$, we can define the cumulative energy $E_r = \sum_{i \leq r} \sigma_i^2 \big/ \sum_i \sigma_i^2$, which is the share of the total that the first r layers keep. The relative error is then

$$\frac{\|A - A_r\|_{\mathrm{F}}}{\|A\|_{\mathrm{F}}} = \sqrt{1 - E_r}.$$

This links the plot of the singular values (Experiment 3) to the quality of the reconstructed image. We use this formula again in the results.

Storage

To save A_r we do not store the whole matrix. We store the r columns u_i (m numbers each), the r rows $v_i^\top$ (n numbers each) and the r singular values:

$$S_{\text{orig}} = mn, \qquad S_{\text{SVD}}(r) = r(m+n+1), \qquad \rho(r) = \frac{mn}{r(m+n+1)}.$$

For Image 1 ($m = 1192$, $n = 1200$) every layer costs 2393 numbers, so $r = 20$ needs 47 860 numbers instead of 1 430 400, a ratio of $\rho = 29.9$. If r gets too large the SVD form needs more numbers than the image itself. This happens when $r > r^* = mn/(m+n+1)$, the break-even rank.

What the code computes

The table lists the formulas behind each function. The code also checks that A is a finite numeric 2-D matrix and that r is a positive integer with $r \le \min(m, n)$, and raises an error if not. Reconstructed pixel values are clipped to $[0, 255]$ before they are shown or saved.

Code	Formula
`load_image`	$\text{gray} = 0.299R + 0.587G + 0.114B$
`low_rank_approximation`	$A_r = U_r \Sigma_r V_r^\top$, written as `(U[:,:r]*S[:r]) @ Vt[:r,:]`
`frobenius_error`	$\|A - A_r\|_{\text{F}} = \left(\sum_{ij}(a_{ij} - (A_r)_{ij})^2\right)^{1/2}$
`relative_error`	$\|A - A_r\|_{\text{F}} / \|A\|_{\text{F}}$
`storage_original`	mn
`storage_svd`	$r(m+n+1)$
`compression_ratio`	$mn \,/\, r(m+n+1)$

Setup. Both images are turned into grayscale and we use $r = 5, 20$ and 100. Image 1 is 1192×1200 ($mn = 1\,430\,400$, $r^* = 597.7$) and shows a garden with hedges, trees and a peacock. Image 2 is 2848×4272 ($mn = 12\,166\,656$, $r^* = 1708.6$) and shows a person using a camera in front of a blurred background.

Results

Experiment 1: effect of rank (Image 1)

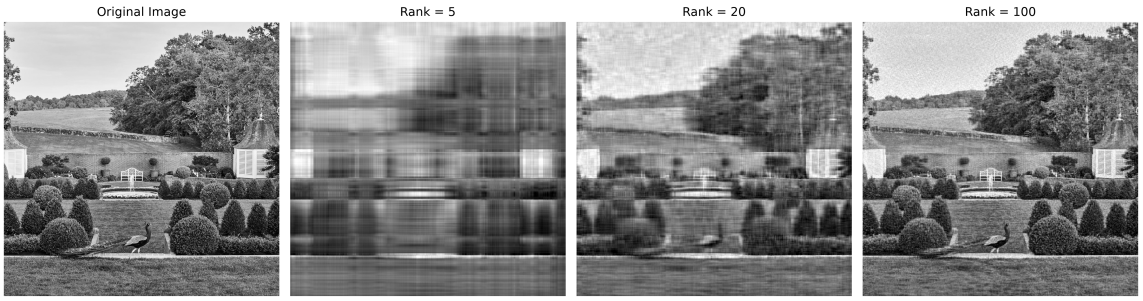


Figure 1: Image 1: the original (before) and the rank-r reconstructions (after).

- $r = 5$**:** we only see the rough light and dark areas,nothing can be recognised, and the picture is full of streaks.
- $r = 20$**:** the tree and few surrounding components are clear. The bench, fountain and peacock are only faintly visible. Leaves and grass have no texture visible.
- $r = 100$**:** almost everything is back, including the peacock. Only the fine detail in the leaves is missing.

The error maps show that smooth parts like the sky and the lawn are easy to store. Most of the error is in the leaves and other fine components, even at $r = 100$.

Experiment 2: error versus storage (Image 1)

r	Frobenius error	Relative error	Original storage	SVD storage	Ratio ρ
5	45 957.9	0.2715	1 430 400	11 965	119.55
20	37 383.2	0.2208	1 430 400	47 860	29.89
100	26 485.9	0.1565	1 430 400	239 300	5.98

Table 1: Error and storage for Image 1.

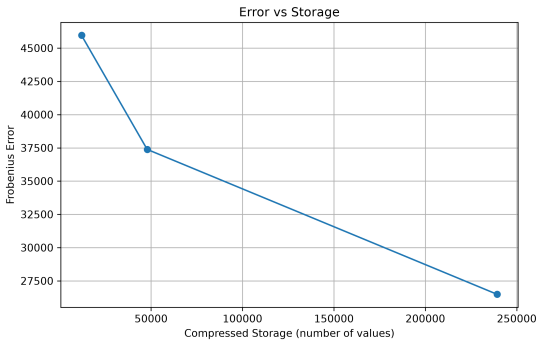


Figure 2: Frobenius error against storage for $r = 5, 20, 100$ (Image 1)

Storage grows in a straight line with r (2393 more numbers for every extra layer), but the error does not fall nearly as fast. Going from $r = 5$ to $r = 20$ lowers the error by 18.7%. Going from $r = 20$ to $r = 100$ needs five times more storage and lowers it by only 29.1% more. At $r = 100$ we store 16.7% of the original size, and the relative error is still 15.6%. So for this image better quality gets expensive quickly.

Experiment 3: singular-value decay (Image 1)

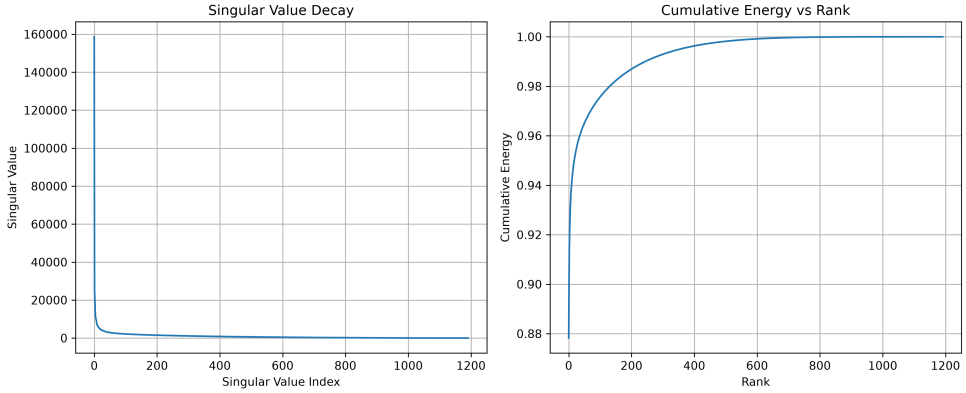


Figure 3: Singular values (left) and cumulative energy E_r (right) for Image 1.

The singular values fall very fast at the start: $\sigma_1 = 158\,645$, $\sigma_5 = 10\,938$, $\sigma_{20} = 4\,801$ and $\sigma_{100} = 2\,157$. The first layer alone already holds $E_1 = 87.8\%$ of the energy. This is mostly the average brightness of the picture, since all pixel values are positive.

At $r = 5$ the energy is $E_5 = 0.926$, which sounds high, but the missing 7.4% gives a relative error of $\sqrt{0.0737} = 0.271$. That is the poor picture we saw in Figure 1. After the first few layers the singular values decrease slowly, and we need $r = 245$ to reach 99% of the energy. So the energy alone makes the quality look better than it is. The relative error is a more honest number.

Experiment 4: second image



Figure 4: Image 2: the original (before) and the rank-r reconstructions (after) as Figure 1.

Image	r	Frobenius error	Relative error	SVD storage	Ratio ρ	E_r
Image 1	5	45 945.8	0.2714	11 965	119.5	0.9263
	20	37 365.1	0.2207	47 860	29.9	0.9512
	100	26 439.0	0.1562	239 300	6.0	0.9755
Image 2	5	49 780.4	0.0964	35 605	341.7	0.9907
	20	23 916.0	0.0463	142 420	85.4	0.9979
	100	8 662.5	0.0168	712 100	17.1	0.9997

Table 2: Both images. Here the errors are measured after clipping to $[0, 255]$, so they are a little smaller than the unclipped values in the Experiment 2 table.

- With the same rank, Image 2 is reconstructed much better. At $r = 100$ its relative error is 0.017 and for Image 1 it is 0.156, which is about nine times larger. Image 2 has 99% of its energy at $r = 5$, while Image 1 needs $r = 245$.
- The singular values of Image 2 fall faster. At index 100 we have $\sigma_{100}/\sigma_1 = 0.0021$ for Image 2 and 0.0136 for Image 1.
- In Image 2 the error is only on sharp edges (camera, hand, hair), because the blurred background is almost exactly low rank. Image 1 has fine texture nearly everywhere. Part of the better compression ratio of Image 2 also comes from its larger size, since a bigger image needs fewer numbers per pixel for the same r.

Conclusions

- A higher rank always gives a smaller error, but the storage grows in a straight line and the gain gets smaller each time. For Image 1, using five times more storage ($r = 20$ to $r = 100$) only brought the relative error from 22.1% down to 15.6%.
- The singular values tell us the quality in advance, because the relative error is $\sqrt{1 - E_r}$. The relative error is a better guide than the energy alone.
- How well the method works depends a lot on the image. The smooth Image 2 gave a ratio of 85 with only 4.6% error at $r = 20$. The detailed Image 1 needed $r = 245$ for 10% error, and that only saves a factor of 2.4.
- Truncated SVD only saves space below the break-even rank ($r \leq 597$ for Image 1 and $r \leq 1708$ for Image 2), so useful ranks are much smaller than $\min(m, n)$.